

Regular Expressions in Python and Bash

Detailed Guide with Many Short and Advanced Examples

Table of Contents

1. [Regex Fundamentals](#)
 2. [Literal Characters](#)
 3. [The Dot Metacharacter](#)
 4. [Character Classes](#)
 5. [Negated Character Classes](#)
 6. [Predefined Character Classes](#)
 7. [Anchors](#)
 8. [Word Boundaries](#)
 9. [Quantifiers](#)
 10. [Greedy and Lazy Quantifiers](#)
 11. [Alternation](#)
 12. [Capturing Groups](#)
 13. [Non-Capturing Groups](#)
 14. [Named Groups](#)
 15. [Backreferences](#)
 16. [Python Regex Functions](#)
 17. [Python Regex Flags](#)
 18. [Lookahead](#)
 19. [Lookbehind](#)
 20. [Conditional Groups](#)
 21. [Atomic Groups](#)
 22. [Possessive Quantifiers](#)
 23. [Overlapping Matches](#)
 24. [Advanced Validation Examples](#)
 25. [Advanced Extraction Examples](#)
 26. [Advanced Replacement Examples](#)
 27. [Regex in Bash](#)
 28. `grep` [Regex Examples](#)
 29. `sed` [Regex Examples](#)
 30. `awk` [Regex Examples](#)
 31. [Real-World Advanced Examples](#)
 32. [Regex Performance](#)
 33. [Common Mistakes](#)
 34. [Quick Reference](#)
-

1. Regex Fundamentals

A **regular expression** is a pattern used to search, validate, extract, split, or replace text.

Example pattern:

```
[0-9]+
```

Meaning:

- `[0-9]` matches one digit.
- `+` means one or more occurrences.

Python:

```
import re

print(re.findall(r"[0-9]+", "Order 125 costs 900"))
```

Output:

```
['125', '900']
```

Bash:

```
echo "Order 125 costs 900" | grep -Eo '[0-9]+'
```

Output:

```
125
900
```

2. Literal Characters

Normal characters usually match themselves.

Pattern:

```
python
```

Python:

```
import re

print(re.findall(r"python", "python and python3"))
```

Output:

```
['python', 'python']
```

The pattern matches `python` even when it appears inside a larger word such as `python3`.

Example: case-sensitive matching

```
import re

print(re.findall(r"Python", "Python python PYTHON"))
```

Output:

```
['Python']
```

Regex matching is case-sensitive by default.

Example: case-insensitive matching

```
import re

print(re.findall(r"python", "Python python PYTHON", re.I))
```

Output:

```
['Python', 'python', 'PYTHON']
```

`re.I` means ignore uppercase and lowercase differences.

3. The Dot Metacharacter

The dot:

```
.
```

matches any single character except a newline by default.

```
import re

print(re.findall(r"c.t", "cat cot cut c9t coat"))
```

Output:

```
['cat', 'cot', 'cut', 'c9t']
```

Explanation:

- `c` matches the letter `c`.
- `.` matches exactly one character.
- `t` matches the letter `t`.

`coat` does not match because it contains two characters between `c` and `t`.

Example: matching file-like names

```
import re

print(re.findall(r"file.", "file1 fileA file_ file"))
```

Output:

```
['file1', 'fileA', 'file_']
```

The dot requires one character after `file`.

Matching a literal dot

Use:

```
\.
```

Example:

```
import re

print(re.findall(r"\.", "version 3.14.2"))
```

Output:

```
['.', '.']
```

Example: match a `.txt` ending

```
import re

print(bool(re.search(r"\.txt$", "notes.txt")))
```

Output:

```
True
```

Explanation:

- `\.` matches a real dot.
- `txt` matches the letters.
- `$` requires the pattern at the end.

4. Character Classes

A character class matches one character from a collection.

Pattern:

```
[abc]
```

Python:

```
import re

print(re.findall(r"[abc]", "apple banana cherry"))
```

Output:

```
['a', 'b', 'a', 'a', 'c']
```

Example: lowercase letters

```
import re  
  
print(re.findall(r"[a-z]", "A1b2C3d"))
```

Output:

```
['b', 'd']
```

Example: uppercase letters

```
import re  
  
print(re.findall(r"[A-Z]", "Hello WORLD"))
```

Output:

```
['H', 'W', 'O', 'R', 'L', 'D']
```

Example: digits

```
import re  
  
print(re.findall(r"[0-9]", "A12B34"))
```

Output:

```
['1', '2', '3', '4']
```

Example: letters and numbers

```
import re

print(re.findall(r"[A-Za-z0-9]", "User_42!"))
```

Output:

```
['U', 's', 'e', 'r', '4', '2']
```

The underscore and exclamation mark are not included.

Example: hexadecimal characters

```
import re

print(re.findall(r"[0-9A-Fa-f]+", "Color: FF20A9"))
```

Output:

```
['C', 'FF20A9']
```

This also matches the `C` in `Color` because `C` is a valid hexadecimal character.

A more specific pattern is:

```
import re

print(re.findall(r"\b[0-9A-Fa-f]{6}\b", "Color: FF20A9"))
```

Output:

```
['FF20A9']
```

5. Negated Character Classes

A caret inside a character class means “anything except these characters.”

Pattern:

```
[^0-9]
```

Example:

```
import re

print(re.findall(r"[^0-9]", "A1B2C3"))
```

Output:

```
['A', 'B', 'C']
```

Example: extract everything except vowels

```
import re

print(re.findall(r"[^aeiou]", "hello"))
```

Output:

```
['h', 'l', 'l']
```

Example: match until a comma

```
import re

print(re.findall(r"[^,]+", "apple,banana,mango"))
```

Output:

```
['apple', 'banana', 'mango']
```

Explanation:

- `[^,]` matches anything except a comma.
 - `+` matches one or more such characters.
-

Example: content inside double quotes

```
import re

print(re.findall(r'"[^"]*"', '"one" "two"'))
```

Output:

```
['"one"', '"two"']
```

`[^"]*` means zero or more characters that are not double quotes.

6. Predefined Character Classes

Python supports shorthand classes.

Pattern	Meaning
<code>\d</code>	Digit
<code>\D</code>	Not a digit
<code>\w</code>	Letter, number, or underscore
<code>\W</code>	Not a word character
<code>\s</code>	Whitespace
<code>\S</code>	Not whitespace

Example: `\d`

```
import re

print(re.findall(r"\d+", "Room 25, floor 3"))
```

Output:

```
['25', '3']
```

Example: `\D`

```
import re

print(re.findall(r"\D+", "123ABC456"))
```

Output:

```
['ABC']
```

Example: `\w`

```
import re

print(re.findall(r"\w+", "hello-world user_name"))
```

Output:

```
['hello', 'world', 'user_name']
```

A hyphen is not a word character, but an underscore is.

Example: `\W`

```
import re

print(re.findall(r"\W", "hello-world!"))
```

Output:

```
['-', '!']
```

Example: `\s`

```
import re

print(re.findall(r"\s", "one two\tthree"))
```

Output conceptually:

```
[' ', '\t']
```

Example: `\S`

```
import re

print(re.findall(r"\S+", "one two three"))
```

Output:

```
['one', 'two', 'three']
```

POSIX classes in Bash

Bash tools commonly use:

```
[:digit:]
[:alpha:]
[:alnum:]
[:space:]
[:upper:]
[:lower:]
```

Example:

```
echo "ID: 4821" | grep -Eo '[:digit:]+'
```

Output:

```
4821
```

7. Anchors

Anchors match positions.

Anchor	Meaning
<code>^</code>	Beginning
<code>\$</code>	End
<code>\A</code>	Absolute beginning in Python
<code>\Z</code>	Absolute end in Python

Beginning of string

```
import re

print(bool(re.search(r"^Hello", "Hello world")))
print(bool(re.search(r"^Hello", "Say Hello")))
```

Output:

```
True
False
```

End of string

```
import re

print(bool(re.search(r"world$", "Hello world")))
print(bool(re.search(r"world$", "world is large")))
```

Output:

```
True
False
```

Match exactly four digits

```
import re

pattern = r"^[0-9]{4}$"
```

```
print(bool(re.search(pattern, "2026")))
print(bool(re.search(pattern, "Year 2026")))
```

Output:

```
True
False
```

Better Python validation with `fullmatch`

```
import re

print(bool(re.fullmatch(r"[0-9]{4}", "2026")))
print(bool(re.fullmatch(r"[0-9]{4}", "Year 2026")))
```

Output:

```
True
False
```

`fullmatch()` requires the entire string to follow the pattern.

8. Word Boundaries

A word boundary is written as:

```
\b
```

Match the complete word `cat`

```
import re

print(re.findall(r"\bcat\b", "cat category wildcat cat"))
```

Output:

```
['cat', 'cat']
```

It does not match `cat` inside `category` or `wildcat`.

Match words beginning with `pro`

```
import re

print(re.findall(r"\bpro\w*", "program improve project"))
```

Output:

```
['program', 'project']
```

`improve` contains `pro`, but it does not begin at a word boundary.

Match words ending with `ing`

```
import re

print(re.findall(r"\b\w+ing\b", "running walked coding"))
```

Output:

```
['running', 'coding']
```

Non-word boundary

Use:

```
\B
```

Example:

```
import re

print(re.findall(r"cat\B", "cat category wildcat"))
```

Output:

```
['cat']
```

This matches `cat` inside `category`, where `cat` is not followed by a word boundary.

9. Quantifiers

Quantifier	Meaning
<code>*</code>	Zero or more
<code>+</code>	One or more
<code>?</code>	Zero or one
<code>{n}</code>	Exactly <code>n</code>
<code>{n,}</code>	At least <code>n</code>
<code>{n,m}</code>	Between <code>n</code> and <code>m</code>

Zero or more

```
import re

print(re.findall(r"ab*", "a ab abb abbb"))
```

Output:

```
['a', 'ab', 'abb', 'abbb']
```

One or more

```
import re

print(re.findall(r"ab+", "a ab abb abbb"))
```

Output:

```
['ab', 'abb', 'abbb']
```

The first `a` does not match because at least one `b` is required.

Zero or one

```
import re

print(re.findall(r"colou?r", "color colour colour"))
```

Output:

```
['color', 'colour']
```

The `u` may appear zero or one time.

Exactly three digits

```
import re

print(re.findall(r"\b[0-9]{3}\b", "12 123 1234"))
```

Output:

```
['123']
```

At least two digits

```
import re

print(re.findall(r"\b[0-9]{2,}\b", "1 12 123 1234"))
```

Output:

```
['12', '123', '1234']
```

Between two and four digits

```
import re

print(re.findall(r"\b[0-9]{2,4}\b", "1 12 123 1234 12345"))
```

Output:

```
['12', '123', '1234']
```

Match repeated letters

```
import re

print(re.findall(r"o{2,}", "book food moon dog"))
```

Output:

```
['oo', 'oo', 'oo']
```

10. Greedy and Lazy Quantifiers

Quantifiers are greedy by default.

Greedy matching

```
import re

text = "<b>one</b><b>two</b>"

print(re.findall(r"<b>.*</b>", text))
```

Output:

```
['<b>one</b><b>two</b>']
```

The `.*` matches as much as possible.

Lazy matching

```
import re

text = "<b>one</b><b>two</b>"

print(re.findall(r"<b>.*?</b>", text))
```

Output:

```
['<b>one</b>', '<b>two</b>']
```

The `*?` matches as little as possible.

Greedy number matching

```
import re

print(re.search(r"[0-9]+", "12345").group())
```

Output:

```
12345
```

Lazy number matching

```
import re

print(re.search(r"[0-9]+?", "12345").group())
```

Output:

```
1
```

`+?` matches the minimum number of characters required.

Better alternative to lazy dot matching

Instead of:

```
".*?"
```

use:

```
"[^"]*"
```

Example:

```
import re  
  
print(re.findall(r'"[^"]*"', '"red" "blue"'))
```

Output:

```
['"red"', '"blue"']
```

This is often clearer and more efficient.

11. Alternation

The pipe means OR.

```
cat|dog
```

Example:

```
import re  
  
print(re.findall(r"cat|dog", "cat dog bird"))
```

Output:

```
['cat', 'dog']
```

Match several file extensions

```
import re
```

```
files = "main.py script.sh notes.txt image.png"
print(re.findall(r"\w+\.(?:py|sh|txt)", files))
```

Output:

```
['main.py', 'script.sh', 'notes.txt']
```

Alternation order matters in Python

```
import re
print(re.search(r"program|programming", "programming").group())
```

Output:

```
program
```

Python accepts the first successful alternative.

Reverse the order:

```
import re
print(re.search(r"programming|program", "programming").group())
```

Output:

```
programming
```

Factor common text

Instead of:

```
colour|color
```

use:

```
colou?r
```

Instead of:

```
program|programming
```

use:

```
program(?:ming)?
```

12. Capturing Groups

Parentheses capture matching text.

```
import re

match = re.fullmatch(r"([A-Z]+)-([0-9]+)", "EMP-4821")

print(match.group(0))
print(match.group(1))
print(match.group(2))
```

Output:

```
EMP-4821
EMP
4821
```

Extract date parts

```
import re

match = re.fullmatch(
    r"([0-9]{4})-([0-9]{2})-([0-9]{2})",
    "2026-07-15",
)

print(match.groups())
```

Output:

```
('2026', '07', '15')
```

Extract name and age

```
import re

match = re.search(r"Name: ([A-Za-z]+), Age: ([0-9]+)", "Name: Arko, Age: 25")

print(match.group(1))
print(match.group(2))
```

Output:

```
Arko
25
```

`findall()` with groups

```
import re

print(re.findall(r"([A-Za-z]+)=([0-9]+)", "width=20 height=10"))
```

Output:

```
[('width', '20'), ('height', '10')]
```

When multiple capturing groups exist, `findall()` returns tuples.

13. Non-Capturing Groups

A non-capturing group is:

```
(?:...)
```

Example:

```
import re

print(re.findall(r"(?:cat|dog)s?", "cats dog birds"))
```

Output:

```
['cats', 'dog']
```

The parentheses group the alternatives but do not save a captured group.

Why non-capturing groups matter

Capturing version:

```
import re

print(re.findall(r"(cat|dog)s?", "cats dogs"))
```

Output:

```
['cat', 'dog']
```

`findall()` returns only the captured group.

Non-capturing version:

```
import re

print(re.findall(r"(?:cat|dog)s?", "cats dogs"))
```

Output:

```
['cats', 'dogs']
```

14. Named Groups

Named groups make patterns easier to understand.

Syntax:

```
(?P<name>...)
```

Example:

```
import re

match = re.fullmatch(
    r"(?P<year>[0-9]{4})-(?P<month>[0-9]{2})-(?P<day>[0-9]{2})",
    "2026-07-15",
)

print(match.group("year"))
print(match.group("month"))
print(match.group("day"))
```

Output:

```
2026
07
15
```

Get all named groups

```
import re

match = re.fullmatch(
    r"(?P<name>[A-Za-z]+):(?P<score>[0-9]+)",
    "Arko:95",
)

print(match.groupdict())
```

Output:

```
{'name': 'Arko', 'score': '95'}
```

Named groups in log parsing

```
import re

line = "level=ERROR code=503"
```



```
match = re.fullmatch(
    r"level=(?P<level>[A-Z]+) code=(?P<code>[0-9]+)",
    line,
)

print(match.groupdict())
```

Output:

```
{'level': 'ERROR', 'code': '503'}
```

15. Backreferences

A backreference matches text captured earlier.

Detect repeated words

```
import re

text = "This is is a test test."

for match in re.finditer(r"\b([A-Za-z]+)\s+\1\b", text):
    print(match.group())
```

Output:

```
is is
test test
```

Explanation:

- `([A-Za-z]+)` captures a word.
 - `\s+` matches spaces.
 - `\1` requires the exact same word again.
-

Match repeated digits

```
import re

print(re.findall(r"([0-9])\1", "112 345 778"))
```

Output:

```
['1', '7']
```

The complete matches are `11` and `77`, but `findall()` returns the captured digits.

To print complete matches:

```
import re

print([m.group() for m in re.finditer(r"([0-9])\1", "112 345 778")])
```

Output:

```
['11', '77']
```

Match repeated two-character blocks

```
import re

print([m.group() for m in re.finditer(r"([A-Za-z]{2})\1", "abab test haha")])
```

Output:

```
['abab', 'haha']
```

Match matching quotation marks

```
import re

text = '''"hello" 'world' "python" '''
```

```
print(re.findall(r'(["\'])(.*?)\1', text))
```

Output:

```
[('\'', 'hello'), ('\'', 'world')]
```

The closing quotation mark must be the same as the opening quotation mark.

16. Python Regex Functions

16.1 `re.search()`

Searches anywhere in the string.

```
import re

match = re.search(r"[0-9]+", "Order number 4821")

print(match.group())
```

Output:

```
4821
```

16.2 `re.match()`

Checks only the beginning.

```
import re

print(bool(re.match(r"Python", "Python regex")))
print(bool(re.match(r"regex", "Python regex")))
```

Output:

```
True
False
```

16.3 re.fullmatch()

Checks the complete string.

```
import re

print(bool(re.fullmatch(r"[A-Z]{3}", "ABC")))
print(bool(re.fullmatch(r"[A-Z]{3}", "ABC123")))
```

Output:

```
True
False
```

16.4 re.findall()

Returns all non-overlapping matches.

```
import re

print(re.findall(r"[0-9]+", "10 apples and 20 mangoes"))
```

Output:

```
['10', '20']
```

16.5 re.finditer()

Returns match objects.

```
import re

for match in re.finditer(r"[A-Z]+", "ABC 123 XYZ"):
    print(match.group(), match.span())
```

Output:

```
ABC (0, 3)
XYZ (8, 11)
```

16.6 re.split()

```
import re

print(re.split(r"[;, ]+", "apple,banana;orange mango"))
```

Output:

```
['apple', 'banana', 'orange', 'mango']
```

Split on one or more spaces

```
import re

print(re.split(r"\s+", "one  two\tthree"))
```

Output:

```
['one', 'two', 'three']
```

Keep separators using a capturing group

```
import re

print(re.split(r"([;,])", "apple,banana;orange"))
```

Output:

```
['apple', ',', 'banana', ';', 'orange']
```

16.7 re.sub()

```
import re

print(re.sub(r"[0-9]+", "NUMBER", "A12 B345"))
```

Output:

```
ANUMBER BNUMBER
```

Replace only the first match

```
import re

print(re.sub(r"cat", "dog", "cat cat cat", count=1))
```

Output:

```
dog cat cat
```

16.8 re.subn()

```
import re

result, count = re.subn(r"cat", "dog", "cat cat bird")

print(result)
print(count)
```

Output:

```
dog dog bird
2
```

16.9 re.compile()

```
import re

pattern = re.compile(r"[0-9]+")

print(pattern.findall("A12 B34"))
```

Output:

```
['12', '34']
```

Compiled patterns are useful when the same regex is reused.

16.10 re.escape()

```
import re

text = "file.txt"
pattern = re.escape(text)

print(pattern)
print(bool(re.search(pattern, "open file.txt")))
```

Output:

```
file\.txt
True
```

`re.escape()` converts regex-special characters into literal characters.

17. Python Regex Flags

17.1 Ignore case

```
import re

print(re.findall(r"error", "ERROR Error error", re.I))
```

Output:

```
['ERROR', 'Error', 'error']
```

17.2 Multiline mode

```
import re

text = "INFO start\nERROR failed\nINFO end"

print(re.findall(r"^ERROR.*$", text, re.M))
```

Output:

```
['ERROR failed']
```

`re.M` allows `^` and `$` to work for every line.

17.3 DOTALL mode

```
import re

text = "hello\nworld"

print(bool(re.search(r"hello.world", text)))
print(bool(re.search(r"hello.world", text, re.S)))
```

Output:

```
False
True
```

`re.S` allows the dot to match a newline.

17.4 Verbose mode

```
import re
```



```

pattern = re.compile(
    r"""
    [A-Za-z0-9._%+-]+
    @
    [A-Za-z0-9.-]+
    \.
    [A-Za-z]{2,}
    """,
    re.X,
)

print(bool(pattern.fullmatch("arko@example.com")))

```

Output:

```
True
```

`re.X` allows spaces, newlines, and comments inside the pattern.

17.5 ASCII mode

```

import re

text = "English বাংলা 123"

print(re.findall(r"\w+", text))
print(re.findall(r"\w+", text, re.A))

```

Output:

```

['English', 'বাংলা', '123']
['English', '123']

```

`re.A` restricts `\w`, `\d`, and related shortcuts to ASCII behavior.

18. Lookahead

Lookahead checks future text without consuming it.

18.1 Positive lookahead

Syntax:

```
(?=...)
```

Extract numbers followed by `INR`:

```
import re

print(re.findall(r"[0-9]+(?=INR)", "500INR 30USD 70INR"))
```

Output:

```
['500', '70']
```

`INR` is required but is not included.

Match words followed by a colon

```
import re

print(re.findall(r"\b[A-Za-z]+(?=:)", "name: Arko age: 25"))
```

Output:

```
['name', 'age']
```

Match filename without extension

```
import re

print(re.findall(r"[A-Za-z0-9_-]+(?=\.\txt)", "notes.txt report.txt"))
```

Output:

```
['notes', 'report']
```

18.2 Negative lookahead

Syntax:

```
(?!...)
```

Match `foo` not followed by `bar`:

```
import re

print(re.findall(r"foo(?!bar)", "foo foobar foo123"))
```

Output:

```
['foo', 'foo']
```

Match files that are not `.exe`

```
import re

files = ["report.txt", "program.exe", "image.png"]

pattern = r"^(?!.*\.exe$).+$"

for file in files:
    if re.fullmatch(pattern, file):
        print(file)
```

Output:

```
report.txt
image.png
```

Password must contain a digit

```
import re

pattern = r"(?=.*[0-9]).{5,}"
```

```
print(bool(re.fullmatch(pattern, "hello1")))  
print(bool(re.fullmatch(pattern, "hello")))
```

Output:

```
True  
False
```

The lookahead checks that at least one digit exists somewhere.

Password must not contain spaces

```
import re  
  
pattern = r"(?!.*\s).+ "  
  
print(bool(re.fullmatch(pattern, "Hello123")))  
print(bool(re.fullmatch(pattern, "Hello 123")))
```

Output:

```
True  
False
```

19. Lookbehind

Lookbehind checks previous text without consuming it.

19.1 Positive lookbehind

Syntax:

```
(?<=...)
```

Extract numbers after ₹ :

```
import re

print(re.findall(r"(?<=₹)[0-9]+", "Book ₹500 Pen ₹25"))
```

Output:

```
['500', '25']
```

Extract username after `user=`

```
import re

print(re.findall(r"(?<=user=)[A-Za-z0-9_]+", "user=arko status=200"))
```

Output:

```
['arko']
```

Extract domain after `@`

```
import re

print(re.findall(r"(?<=@)[A-Za-z0-9.-]+", "arko@example.com"))
```

Output:

```
['example.com']
```

19.2 Negative lookbehind

Syntax:

```
(?<!\...)
```

Extract numbers not preceded by `$`:

```
import re

print(re.findall(r"(?!\\$)\\b[0-9]+\\b", "$100 200 $300 400"))
```

Output:

```
['200', '400']
```

Match `cat` not preceded by `wild`

```
import re

print(re.findall(r"(?!wild)cat", "cat wildcat bobcat"))
```

Output:

```
['cat', 'cat']
```

The final match comes from `bobcat`.

Python lookbehind limitation

This is valid:

```
(?<=abc)
```

This is invalid in Python's built-in `re`:

```
(?<=a+)
```

Lookbehind must normally have a fixed length.

20. Conditional Groups

A conditional group checks whether another capturing group participated.

Syntax:

```
(?(group)yes-pattern|no-pattern)
```

Match optional angle brackets correctly

```
import re

pattern = r"^(<)?[A-Za-z]+(?(1)>|)$"

values = ["Alice", "<Alice>", "<Alice", "Alice>"]

for value in values:
    print(value, bool(re.fullmatch(pattern, value)))
```

Output:

```
Alice True
<Alice> True
<Alice False
Alice> False
```

If the opening `<` is present, the closing `>` is required.

Match optional parentheses around a number

```
import re

pattern = r"^(\\()?[0-9]+(?(1)\\)|)$"

for value in ["123", "(123)", "(123", "123)":
    print(value, bool(re.fullmatch(pattern, value)))
```

Output:

```
123 True
(123) True
(123 False
123) False
```

21. Atomic Groups

Atomic groups are available in Python 3.11 and later.

Syntax:

```
(?>...)
```

An atomic group does not allow backtracking inside it.

Normal greedy matching

```
import re

print(bool(re.fullmatch(r"a*a", "aaaa")))
```

Output:

```
True
```

Explanation:

1. `a*` initially matches every `a`.
 2. The final `a` cannot match.
 3. The regex engine returns one `a`.
 4. The final `a` matches successfully.
-

Atomic matching

```
import re

print(bool(re.fullmatch(r"(?>a*)a", "aaaa")))
```

Output:

```
False
```

The atomic group consumes every `a` and refuses to return one.

Atomic alternation

```
import re

print(bool(re.fullmatch(r"(?>ab|a)b", "ab")))
```

Output:

False

Explanation:

- The atomic group first matches `ab`.
- The final `b` has nothing left.
- The group cannot backtrack and choose `a`.

Without the atomic group:

```
import re

print(bool(re.fullmatch(r"(?:ab|a)b", "ab")))
```

Output:

True

The normal group can backtrack from `ab` to `a`.

22. Possessive Quantifiers

Possessive quantifiers are available in Python 3.11 and later.

Normal	Possessive
<code>*</code>	<code>*+</code>
<code>+</code>	<code>++</code>
<code>?</code>	<code>?+</code>
<code>{n,m}</code>	<code>{n,m}+</code>

They do not backtrack.

Possessive `*+`

```
import re

print(bool(re.fullmatch(r"a*+a", "aaaa")))
```

Output:

```
False
```

`a*+` consumes everything and refuses to return a character.

Normal versus possessive

```
import re

print(bool(re.fullmatch(r"[0-9]*[0-9]", "123")))
print(bool(re.fullmatch(r"[0-9]*+[0-9]", "123")))
```

Output:

```
True
False
```

Possessive quantifier for performance

```
import re

pattern = r"^[A-Za-z]*+"

print(bool(re.fullmatch(pattern, "Python")))
print(bool(re.fullmatch(pattern, "Python123")))
```

Output:

```
True
False
```

The possessive quantifier can prevent unnecessary backtracking.

23. Overlapping Matches

Normal matches do not overlap.

```
import re

print(re.findall(r"AA", "AAAA"))
```

Output:

```
['AA', 'AA']
```

Matches occur at positions:

```
0-2
2-4
```

Find overlapping matches

```
import re

for match in re.finditer(r"(?=AA)", "AAAA"):
    print(match.start(), match.group(1))
```

Output:

```
0 AA
1 AA
2 AA
```

The lookahead consumes no characters, allowing the next match to begin one position later.

Overlapping three-digit sequences

```
import re

text = "12345"

print([
    match.group(1)
```

```
    for match in re.finditer(r"(?=[0-9]{3})", text)
  ])
```

Output:

```
['123', '234', '345']
```

Find repeated overlapping pairs

```
import re

text = "banana"

print([
    match.group(1)
    for match in re.finditer(r"(?=[a-z]{2})", text)
])
```

Output:

```
['ba', 'an', 'na', 'an', 'na']
```

24. Advanced Validation Examples

24.1 Username validation

Rules:

- Starts with a letter
- Contains letters, digits, or
- Length from 3 to 16

```
import re

pattern = r"[A-Za-z][A-Za-z0-9_]{2,15}"

values = ["arko_123", "123arko", "ab", "validUser"]

for value in values:
    print(value, bool(re.fullmatch(pattern, value)))
```

Output:

```
arko_123 True
123arko False
ab False
validUser True
```

24.2 Strong password structure

Rules:

- At least eight characters
- One lowercase letter
- One uppercase letter
- One digit
- One special character
- No whitespace

```
import re

pattern = (
    r"(?=.*[a-z])"
    r"(?=.*[A-Z])"
    r"(?=.*[0-9])"
    r"(?=.*[^\A-Za-z0-9])"
    r"(?!.*\s)"
    r".{8,}"
)

for value in ["Hello123!", "hello123!", "Hello 123!"]:
    print(value, bool(re.fullmatch(pattern, value)))
```

Output:

```
Hello123! True
hello123! False
Hello 123! False
```

24.3 PIN validation

Exactly four or six digits:

```
import re

pattern = r"(?:[0-9]{4}|[0-9]{6})"

for pin in ["1234", "123456", "12345", "12A4"]:
    print(pin, bool(re.fullmatch(pattern, pin)))
```

Output:

```
1234 True
123456 True
12345 False
12A4 False
```

24.4 Indian mobile-number structure

```
import re

pattern = r"(?:\+91[- ])?[6-9][0-9]{9}"

numbers = [
    "9876543210",
    "+91 9876543210",
    "+91-9876543210",
    "1234567890",
]

for number in numbers:
    print(number, bool(re.fullmatch(pattern, number)))
```

Output:

```
9876543210 True
+91 9876543210 True
+91-9876543210 True
1234567890 False
```

24.5 Email-like validation

```
import re
```

```

pattern = r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"

for email in [
    "arko@example.com",
    "user.name@test.co.in",
    "user@example",
    "@example.com",
]:
    print(email, bool(re.fullmatch(pattern, email)))

```

Output:

```

arko@example.com True
user.name@test.co.in True
user@example False
@example.com False

```

This checks a practical structure, not every rule from the complete email standard.

24.6 IPv4 validation

```

import re

octet = r"(:25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])"
pattern = rf"({octet}\.){3}{octet}"

for ip in [
    "192.168.1.1",
    "255.255.255.255",
    "256.1.1.1",
    "192.168.1",
]:
    print(ip, bool(re.fullmatch(pattern, ip)))

```

Output:

```

192.168.1.1 True
255.255.255.255 True
256.1.1.1 False
192.168.1 False

```

24.7 Date-format validation

```
import re

pattern = r"[0-9]{4}-(?:0[1-9]|1[0-2])-(?:0[1-9]|[12][0-9]|3[01])"

for value in ["2026-07-15", "2026-13-10", "2026-02-31"]:
    print(value, bool(re.fullmatch(pattern, value)))
```

Output:

```
2026-07-15 True
2026-13-10 False
2026-02-31 True
```

The regex validates ranges but does not understand that February 31 is impossible.

Use `datetime` for calendar validation.

```
from datetime import datetime

for value in ["2026-07-15", "2026-02-31"]:
    try:
        datetime.strptime(value, "%Y-%m-%d")
        print(value, True)
    except ValueError:
        print(value, False)
```

Output:

```
2026-07-15 True
2026-02-31 False
```

24.8 Hexadecimal color validation

```
import re

pattern = r"#(?:[0-9A-Fa-f]{3}|[0-9A-Fa-f]{6})"

for color in ["#FFF", "#FF20A9", "#GGG", "FF20A9"]:
    print(color, bool(re.fullmatch(pattern, color)))
```


Output:

```
#FFF True
#FF20A9 True
#GGG False
FF20A9 False
```

24.9 Semantic-version structure

```
import re

pattern = (
    r"(0|[1-9][0-9]*)\."
    r"(0|[1-9][0-9]*)\."
    r"(0|[1-9][0-9]*)"
    r"(?:-([0-9A-Za-z.-]+))?"
    r"(?:\+([0-9A-Za-z.-]+))?"
)

for version in ["1.2.3", "2.0.0-beta", "1.0", "01.2.3"]:
    print(version, bool(re.fullmatch(pattern, version)))
```

Output:

```
1.2.3 True
2.0.0-beta True
1.0 False
01.2.3 False
```

24.10 Filename validation

Rules:

- Starts with a letter or number
- May contain `_`, `-`, and `.`
- No spaces

```
import re

pattern = r"[A-Za-z0-9][A-Za-z0-9_.-]*"
```

```
for name in ["report.txt", "my_file-1.py", "my file.txt", ".hidden"]:  
    print(name, bool(re.fullmatch(pattern, name)))
```

Output:

```
report.txt True  
my_file-1.py True  
my file.txt False  
.hidden False
```

25. Advanced Extraction Examples

25.1 Extract hashtags

```
import re  
  
text = "Learning #Python and #Regex today"  
  
print(re.findall(r"#[A-Za-z0-9_]+", text))
```

Output:

```
['#Python', '#Regex']
```

25.2 Extract mentions

```
import re  
  
text = "Contact @arko or @admin_user"  
  
print(re.findall(r"@[A-Za-z0-9_]+", text))
```

Output:

```
['@arko', '@admin_user']
```

25.3 Extract file extensions

```
import re

files = "main.py notes.txt image.png archive.tar.gz"

print(re.findall(r"\.([A-Za-z0-9]+\b)", files))
```

Output:

```
['py', 'txt', 'png', 'tar', 'gz']
```

The pattern captures every extension-like section after a dot.

25.4 Extract final file extensions

```
import re

files = ["main.py", "archive.tar.gz", "README"]

for file in files:
    match = re.search(r"\.([^.]+$)", file)
    print(file, match.group(1) if match else None)
```

Output:

```
main.py py
archive.tar.gz gz
README None
```

25.5 Extract URLs

```
import re

text = "Visit https://example.com and http://test.org/page"

pattern = r"https?://[A-Za-z0-9./?=_-]+"

print(re.findall(pattern, text))
```

Output:

```
['https://example.com', 'http://test.org/page']
```

25.6 Extract domains from email addresses

```
import re

text = "alice@example.com bob@test.org"

print(re.findall(r"@[A-Za-z0-9.-]+", text))
```

Output:

```
['@example.com', '@test.org']
```

Without the `@`:

```
import re

print(re.findall(r"(?<=)[A-Za-z0-9.-]+", text))
```

Output:

```
['example.com', 'test.org']
```

25.7 Extract key-value pairs

```
import re

text = "user=arko status=200 duration=145"

print(re.findall(r"([A-Za-z_]+)=([^\s]+)", text))
```

Output:

```
[('user', 'arko'), ('status', '200'), ('duration', '145')]
```

Convert key-value pairs into a dictionary

```
import re

text = "user=arko status=200 duration=145"

result = dict(re.findall(r"([A-Za-z_]+)=(^[ ]+)", text))

print(result)
```

Output:

```
{'user': 'arko', 'status': '200', 'duration': '145'}
```

25.8 Extract HTML-like tags

```
import re

text = "<h1>Title</h1><p>Paragraph</p>"

print(re.findall(r"</?([A-Za-z0-9]+)>", text))
```

Output:

```
['h1', 'h1', 'p', 'p']
```

This is suitable for simple extraction, not complete HTML parsing.

25.9 Extract content between tags

```
import re

text = "<title>Regex Guide</title>"

match = re.search(r"<title>(.*?)</title>", text)

print(match.group(1))
```

Output:

25.10 Extract quoted strings

```
import re

text = '"Python" "Bash" "Regex"'

print(re.findall(r'"([^\"]*)"', text))
```

Output:

```
['Python', 'Bash', 'Regex']
```

25.11 Extract numbers with units

```
import re

text = "10kg 25cm 5MB"

print(re.findall(r"([0-9]+)([A-Za-z]+)", text))
```

Output:

```
[('10', 'kg'), ('25', 'cm'), ('5', 'MB')]
```

25.12 Extract protocol, host, and path

```
import re

url = "https://example.com/docs/page"

match = re.fullmatch(
    r"(https?)://([^\s/]+)(/.*)?",
    url,
)
```

```
print(match.groups())
```

Output:

```
('https', 'example.com', '/docs/page')
```

25.13 Extract time values

```
import re

text = "Meeting at 09:30 and lunch at 13:15"

print(re.findall(r"\b[0-2][0-9]:[0-5][0-9]\b", text))
```

Output:

```
['09:30', '13:15']
```

This also accepts `29:30`, so a more accurate hour pattern is:

```
import re

pattern = r"\b(?:[01][0-9]|2[0-3]):[0-5][0-9]\b"

print(re.findall(pattern, "09:30 23:59 29:30"))
```

Output:

```
['09:30', '23:59']
```

25.14 Extract Markdown headings

```
import re

text = "# Main\n## Section\nNormal text"

print(re.findall(r"^(#{1,6})\s+(.+)$", text, re.M))
```

Output:

```
['Main', 'Section']
```

25.15 Extract Markdown links

```
import re

text = "[Python](https://python.org) and [Bash](https://gnu.org)"

print(re.findall(r"\[([^\]]+)\]\([^\)]+\)", text))
```

Output:

```
[('Python', 'https://python.org'), ('Bash', 'https://gnu.org')]
```

26. Advanced Replacement Examples

26.1 Reformat a date

```
import re

date = "2026-07-15"

result = re.sub(
    r"([0-9]{4})-([0-9]{2})-([0-9]{2})",
    r"\3/\2/\1",
    date,
)

print(result)
```

Output:

```
15/07/2026
```

26.2 Swap first and last names

```
import re

name = "Arko Mukherjee"

print(re.sub(r"([A-Za-z]+) ([A-Za-z]+)", r"\2, \1", name))
```

Output:

```
Mukherjee, Arko
```

26.3 Mask phone numbers

```
import re

phone = "9876543210"

print(re.sub(r"[0-9](?=[0-9]{4})", "*", phone))
```

Output:

```
*****3210
```

The lookahead preserves the last four digits.

26.4 Mask an email username

```
import re

email = "arko@example.com"

print(re.sub(r"(?<=^.).*(?=@)", "***", email))
```

Output:

```
a***@example.com
```

Explanation:

- `(?<=^.)` requires one character at the beginning.
 - `.*` matches the remaining username.
 - `(?=@)` stops before the `@`.
-

26.5 Convert repeated spaces to one space

```
import re

text = "Python    regex    is useful"

print(re.sub(r"\s+", " ", text))
```

Output:

```
Python regex is useful
```

26.6 Remove leading and trailing whitespace

```
import re

text = "    hello world    "

print(re.sub(r"^\s+|\s+$", "", text))
```

Output:

```
hello world
```

Python's `strip()` is simpler for this exact task.

26.7 Add commas to numbers

```
import re

number = "123456789"

result = re.sub(r"(?<=\d)(?=(\d{3})+)$", ",", number)
```

```
print(result)
```

Output:

```
123,456,789
```

The regex identifies positions where groups of three digits remain.

26.8 Replace words using a function

```
import re

def uppercase(match):
    return match.group().upper()

text = "python bash regex"

print(re.sub(r"\bpython|bash\b", uppercase, text))
```

Output:

```
PYTHON BASH regex
```

26.9 Double every number

```
import re

text = "10 apples and 20 mangoes"

result = re.sub(
    r"[0-9]+",
    lambda match: str(int(match.group()) * 2),
    text,
)

print(result)
```

Output:

20 apples and 40 mangoes

26.10 Remove duplicate words

```
import re

text = "This is is a test test."

result = re.sub(
    r"\b([A-Za-z]+)(\s+\1)+\b",
    r"\1",
    text,
    flags=re.I,
)

print(result)
```

Output:

This is a test.

27. Regex in Bash

Bash regex matching uses:

```
[[ string =~ regex ]]
```

Simple number check

```
value="12345"

if [[ $value =~ ^[0-9]+$ ]]; then
    echo "Only digits"
fi
```

Output:

Only digits

Contains a number

```
value="Order4821"

if [[ $value =~ [0-9]+ ]]; then
    echo "Contains a number"
fi
```

Output:

Contains a number

Validate a username

```
username="arko_123"
regex='^[A-Za-z][A-Za-z0-9_]{2,15}$'

if [[ $username =~ $regex ]]; then
    echo "Valid username"
fi
```

Output:

Valid username

Capture date parts

```
value="2026-07-15"
regex='^([0-9]{4})-([0-9]{2})-([0-9]{2})$'

if [[ $value =~ $regex ]]; then
    echo "Year: ${BASH_REMATCH[1]}"
    echo "Month: ${BASH_REMATCH[2]}"
    echo "Day: ${BASH_REMATCH[3]}"
fi
```

Output:

```
Year: 2026
Month: 07
Day: 15
```

Capture an employee ID

```
value="EMP-4821"
regex='^([A-Z]+)-([0-9]+)$'

if [[ $value =~ $regex ]]; then
    echo "Prefix: ${BASH_REMATCH[1]}"
    echo "Number: ${BASH_REMATCH[2]}"
fi
```

Output:

```
Prefix: EMP
Number: 4821
```

Validate an email-like structure

```
email="ariko@example.com"
regex='^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'

if [[ $email =~ $regex ]]; then
    echo "Valid email structure"
fi
```

Output:

```
Valid email structure
```

Match a `.txt` extension

```
filename="notes.txt"
regex='[.]txt$'
```

```
if [[ $filename =~ $regex ]]; then
    echo "Text file"
fi
```

Output:

```
Text file
```

Using `[.]` avoids some backslash-quoting confusion.

Case-insensitive matching

```
shopt -s nocasematch

value="PYTHON"

if [[ $value =~ ^python$ ]]; then
    echo "Matched"
fi

shopt -u nocasematch
```

Output:

```
Matched
```

Capture URL parts

```
url="https://example.com/docs"
regex='^(https?)://([^\s/]+)(/.*)?$'

if [[ $url =~ $regex ]]; then
    echo "Protocol: ${BASH_REMATCH[1]}"
    echo "Host: ${BASH_REMATCH[2]}"
    echo "Path: ${BASH_REMATCH[3]}"
fi
```

Output:

```
Protocol: https
Host: example.com
Path: /docs
```

Bash regex-variable quoting

Correct:

```
value="123"
regex='^[0-9]+$'

[[ $value =~ $regex ]] && echo "Matched"
```

Output:

```
Matched
```

Avoid:

```
[[ $value =~ "$regex" ]]
```

Quoting the entire regex variable causes Bash to treat it more literally.

28. `grep` Regex Examples

28.1 Search for text

```
echo "ERROR Database failed" | grep 'ERROR'
```

Output:

```
ERROR Database failed
```

28.2 Ignore case

```
printf '%s\n' ERROR Error error INFO | grep -i '^error$'
```

Output:

```
ERROR
Error
error
```

28.3 Match complete words

```
echo "cat category wildcat" | grep -oE '\bcat\b'
```

Output:

```
cat
```

Depending on the `grep` implementation, word boundaries may also be written using `-w`:

```
echo "cat category wildcat" | grep -ow 'cat'
```

Output:

```
cat
```

28.4 Print numbers only

```
echo "IDs: 125 900" | grep -Eo '[0-9]+'
```

Output:

```
125
900
```

28.5 Match alternatives

```
printf '%s\n' INFO ERROR WARNING DEBUG |  
grep -E 'ERROR|WARNING'
```

Output:

```
ERROR  
WARNING
```

28.6 Match lines beginning with ERROR

```
printf '%s\n'  
    "ERROR Failed"  
    "An ERROR happened" |  
grep '^ERROR'
```

Output:

```
ERROR Failed
```

28.7 Match lines ending with .com

```
printf '%s\n' example.com test.org sample.com |  
grep '\.com$'
```

Output:

```
example.com  
sample.com
```

28.8 Find email-like addresses

```
echo "Mail alice@example.com and bob@test.org" |  
grep -Eo '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}'
```

Output:

```
alice@example.com  
bob@test.org
```

28.9 Find IPv4-like addresses

```
echo "Server 192.168.1.10 and 10.0.0.5" |  
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}'
```

Output:

```
192.168.1.10  
10.0.0.5
```

28.10 Positive lookbehind with GNU `grep -P`

```
echo "user=arko status=200" |  
grep -Po '(?<=user=)[A-Za-z0-9_]+'
```

Output:

```
arko
```

`grep -P` is GNU/PCRE-style functionality and may not be available everywhere.

28.11 Positive lookahead with `grep -P`

```
echo "500INR 20USD 70INR" |  
grep -Po '[0-9]+(?=INR)'
```

Output:

```
500  
70
```

28.12 Detect repeated words

```
echo "This is is a test test." |  
grep -Poi '\b([A-Za-z]+\s+\1\b)'
```

Output:

```
is is  
test test
```

This requires PCRE-compatible `grep -P`.

28.13 Exclude empty lines

```
printf 'one\n\n\ttwo\n' |  
grep -Ev '^[[:space:]]*$'
```

Output:

```
one  
two
```

28.14 Exclude comment lines

```
printf '%s\n'
    "# comment"
    "value=10"
    " # another comment" |
grep -Ev '^[[:space:]]*#'
```

Output:

```
value=10
```

29. sed Regex Examples

29.1 Replace the first match

```
echo "cat cat cat" | sed 's/cat/dog/'
```

Output:

```
dog cat cat
```

29.2 Replace every match

```
echo "cat cat cat" | sed 's/cat/dog/g'
```

Output:

```
dog dog dog
```

29.3 Replace digits

```
echo "User123 Code456" |  
sed -E 's/[0-9]+/NUMBER/g'
```

Output:

```
UserNUMBER CodeNUMBER
```

29.4 Reformat a date

```
echo "2026-07-15" |  
sed -E 's/^([0-9]{4})-([0-9]{2})-([0-9]{2})$/\3\/\2\/\1/'
```

Output:

```
15/07/2026
```

29.5 Swap names

```
echo "Arko Mukherjee" |  
sed -E 's/^([^\ ]+) ([^\ ]+)$/2, 1/'
```

Output:

```
Mukherjee, Arko
```

29.6 Extract a username

```
echo "user=arko status=200" |  
sed -En 's/.*user=([^\ ]+).*/\1/p'
```

Output:

```
arko
```

Explanation:

- `-E` enables extended regex.
- `-n` disables automatic printing.
- `p` prints only when substitution succeeds.

29.7 Remove HTML-like tags

```
echo '<p>Hello <b>world</b></p>' |  
sed -E 's/<[^>]*>//g'
```

Output:

```
Hello world
```

This is only suitable for simple text, not full HTML parsing.

29.8 Remove leading whitespace

```
echo "    hello" |  
sed 's/^[[:space:]]*//'
```

Output:

```
hello
```

29.9 Remove trailing whitespace

```
printf 'hello    \n' |  
sed 's/[[:space:]]*$//'
```

Output:

```
hello
```

29.10 Replace repeated spaces

```
echo "Python    regex    Bash" |  
sed -E 's/[[:space:]]+/ /g'
```

Output:

```
Python regex Bash
```

29.11 Remove comments

```
echo "PORT=8080 # server port" |  
sed 's/[[:space:]]*#.*$//'
```

Output:

```
PORT=8080
```

29.12 Mask a phone number

```
echo "9876543210" |  
sed -E 's/^([0-9]{6})([0-9]{4})$/*****\2/'
```

Output:

```
*****3210
```

29.13 Add a prefix to each line

```
printf '%s\n' one two |  
sed 's/^/Item: /'
```

Output:

```
Item: one  
Item: two
```

29.14 Add a suffix to each line

```
printf '%s\n' one two |  
sed 's/$/ completed/'
```

Output:

```
one completed  
two completed
```

30. `awk` Regex Examples

30.1 Print lines containing `ERROR`

```
printf '%s\n'  
  "INFO Started"  
  "ERROR Failed" |  
awk '/ERROR/'
```

Output:

```
ERROR Failed
```

30.2 Match the first field

```
echo "ERROR Database failed" |  
awk '$1 ~ /^ERROR$/ { print $0 }'
```

Output:

```
ERROR Database failed
```

30.3 Match a department

```
echo "101 Alice Engineering 75000" |  
awk '$3 ~ /^Engineering$/ { print $2 }'
```

Output:

```
Alice
```

30.4 Negative matching

```
printf '%s\n'  
    "Alice Engineering"  
    "Bob Sales" |  
awk '$2 !~ /Sales/ { print $1 }'
```

Output:

```
Alice
```

30.5 Extract a matching value

```
echo "Order ABC-4821 completed" |  
awk '  
match($0, /[A-Z]+-[0-9]+)/ {
```

```
    print substr($0, RSTART, RLENGTH)
}'
```

Output:

```
ABC-4821
```

30.6 Dynamic regex

```
pattern='^A'

printf '%s\n' Alice Bob Andrew |
awk -v regex="$pattern" '$0 ~ regex'
```

Output:

```
Alice
Andrew
```

30.7 Case-insensitive matching

Portable method:

```
printf '%s\n' ERROR Error error INFO |
awk 'tolower($0) == "error"'
```

Output:

```
ERROR
Error
error
```

30.8 Count matching lines

```
printf '%s\n'
    "ERROR one"
    "INFO two"
```

```
"ERROR three" |  
awk '/ERROR/ { count++ } END { print count }'
```

Output:

```
2
```

30.9 Count values by pattern

```
printf '%s\n'  
  "200 OK"  
  "404 Missing"  
  "200 OK" |  
awk '  
$1 ~ /^2[0-9][0-9]$/ { success++ }  
$1 ~ /^4[0-9][0-9]$/ { client_error++ }  
  
END {  
  print "Success:", success  
  print "Client errors:", client_error  
}'
```

Output:

```
Success: 2  
Client errors: 1
```

30.10 Replace using `gsub()`

```
echo "cat cat dog" |  
awk '{ gsub(/cat/, "tiger"); print }'
```

Output:

```
tiger tiger dog
```

30.11 Remove non-digits

```
echo "Phone: +91-98765-43210" |  
awk '{  
    value = $0  
    gsub(/^[^0-9]/, "", value)  
    print value  
}'
```

Output:

```
919876543210
```

30.12 Split using regex

```
echo "apple,banana;orange" |  
awk '{  
    count = split($0, values, /[;,]/)  
  
    for (i = 1; i <= count; i++) {  
        print values[i]  
    }  
}'
```

Output:

```
apple  
banana  
orange
```

31. Real-World Advanced Examples

31.1 Parse a log line in Python

```
import re  
  
line = "2026-07-15 ERROR user=arko code=503"  
  
pattern = (
```

```

    r"(?P<date>[0-9]{4}-[0-9]{2}-[0-9]{2}) "
    r"(?P<level>[A-Z]+) "
    r"user=(?P<user>[A-Za-z0-9_]+) "
    r"code=(?P<code>[0-9]+)"
)

match = re.fullmatch(pattern, line)

print(match.groupdict())

```

Output:

```
{'date': '2026-07-15', 'level': 'ERROR', 'user': 'arko', 'code': '503'}
```

31.2 Extract all error codes

```

import re

text = """
ERROR code=500
INFO code=200
ERROR code=503
"""

print(re.findall(r"^ERROR code=([0-9]+)$", text, re.M))

```

Output:

```
['500', '503']
```

31.3 Detect duplicate consecutive words

```

import re

text = "The the quick brown fox fox jumped."

print([
    match.group()
    for match in re.finditer(
        r"\b([A-Za-z]+)\s+\1\b",
        text,
        re.I,
    )
])

```

```
)  
])
```

Output:

```
['The the', 'fox fox']
```

31.4 Detect duplicate punctuation

```
import re  
  
text = "What?? Really!! Fine."  
  
print(re.findall(r"([!?])\1+", text))
```

Output:

```
['?', '!']
```

Complete matches:

```
import re  
  
text = "What?? Really!! Fine."  
  
print([  
    match.group()  
    for match in re.finditer(r"([!?])\1+", text)  
)
```

Output:

```
['??', '!!!']
```

31.5 Extract balanced quote types

```
import re  
  
text = '''"Python" 'Bash' "Wrong\''''
```

```
pattern = r'(?P<quote>["\'])(?P<value>.*?)(?P=quote)'  
  
for match in re.finditer(pattern, text):  
    print(match.group("value"))
```

Output:

```
Python  
Bash
```

31.6 Find words containing all five vowels

```
import re  
  
words = ["education", "sequoia", "computer", "facetious"]  
  
pattern = (  
    r"(?=.*a)"  
    r"(?=.*e)"  
    r"(?=.*i)"  
    r"(?=.*o)"  
    r"(?=.*u)"  
    r"[A-Za-z]+"  
)  
  
for word in words:  
    if re.fullmatch(pattern, word, re.I):  
        print(word)
```

Output:

```
education  
sequoia  
facetious
```

Each lookahead requires one vowel somewhere in the word.

31.7 Match words without repeated characters

```
import re
```



```
pattern = r"(?!.*([A-Za-z]).*\1)[A-Za-z]+"

for word in ["lamp", "letter", "world", "hello"]:
    print(word, bool(re.fullmatch(pattern, word, re.I)))
```

Output:

```
lamp True
letter False
world True
hello False
```

The negative lookahead rejects a word when the same character appears again later.

31.8 Match words with a repeated character

```
import re

pattern = r"(?=.*([A-Za-z]).*\1)[A-Za-z]+"

for word in ["lamp", "letter", "world", "hello"]:
    print(word, bool(re.fullmatch(pattern, word, re.I)))
```

Output:

```
lamp False
letter True
world False
hello True
```

31.9 Extract text between markers

```
import re

text = "START first END START second END"

print(re.findall(r"START\s+(.*?)\s+END", text))
```

Output:

```
['first', 'second']
```

31.10 Parse simple assignments

```
import re

text = """
name = Arko
age = 25
language = Python
"""

pairs = re.findall(
    r"^[A-Za-z_][A-Za-z0-9_]*\s*=\s*(.+)$",
    text,
    re.M,
)

print(dict(pairs))
```

Output:

```
{'name': 'Arko', 'age': '25', 'language': 'Python'}
```

31.11 Find suspicious repeated path separators

```
import re

paths = [
    "/home/user/file",
    "/home//user///file",
]

for path in paths:
    print(path, bool(re.search(r"/{2,}", path)))
```

Output:

```
/home/user/file False
/home//user///file True
```

31.12 Normalize repeated path separators

```
import re

path = "/home//user///file"

print(re.sub(r"/{2,}", "/", path))
```

Output:

```
/home/user/file
```

31.13 Parse HTTP request lines

```
import re

line = "GET /api/users HTTP/1.1"

match = re.fullmatch(
    r"(GET|POST|PUT|DELETE|PATCH) ([^ ]+) HTTP/([0-9.]+)",
    line,
)

print(match.groups())
```

Output:

```
('GET', '/api/users', '1.1')
```

31.14 Parse simple Apache-style information

```
import re

line = '192.168.1.10 "GET /index.html" 200'

match = re.fullmatch(
    r"([0-9.]+) \"([A-Z]+) ([^\" ]+)\" ([0-9]{3})",
    line,
)
```

```
print(match.groups())
```

Output:

```
('192.168.1.10', 'GET', '/index.html', '200')
```

31.15 Extract CSV-like values without quoted commas

```
import re

text = "Alice,Engineering,75000"

print(re.findall(r"^[^,]+", text))
```

Output:

```
['Alice', 'Engineering', '75000']
```

This fails for values such as:

```
"Mukherjee, Arko",Engineering,75000
```

Use Python's `csv` module for real CSV data.

31.16 Validate arithmetic-expression shape

```
import re

pattern = r"[0-9]+(?:\s*[+ \- */]\s*[0-9]+)*"

for expression in ["10 + 20", "10*5-2", "10 +", "a + 2"]:
    print(expression, bool(re.fullmatch(pattern, expression)))
```

Output:

```
10 + 20 True
10*5-2 True
10 + False
a + 2 False
```

This checks simple structure but does not calculate the expression.

31.17 Find Python function names

```
import re

source = """
def add(a, b):
    return a + b

def greet(name):
    print(name)
"""

print(re.findall(r"^def\s+([A-Za-z_][A-Za-z0-9_]*)\s*\(", source, re.M))
```

Output:

```
['add', 'greet']
```

For complete Python parsing, use the `ast` module.

31.18 Find Bash variable assignments

```
import re

script = """
name="Arko"
count=10
echo "$name"
"""

print(re.findall(
    r"^[A-Za-z_][A-Za-z0-9_]*=",
    script,
    re.M,
))
```

Output:

```
['name', 'count']
```

31.19 Extract environment-variable references

```
import re

text = 'Home: $HOME User: ${USER} Shell: $SHELL'

pattern = r"\${(?:([A-Za-z_][A-Za-z0-9_]*)|([A-Za-z_][A-Za-z0-9_]*))}"

print(re.findall(pattern, text))
```

Output:

```
(('', 'HOME'), ('USER', ''), ('', 'SHELL'))
```

A cleaner extraction:

```
import re

text = 'Home: $HOME User: ${USER} Shell: $SHELL'

pattern = r"\${(?:[A-Za-z_][A-Za-z0-9_]*)}?"

print(re.findall(pattern, text))
```

Output:

```
['HOME', 'USER', 'SHELL']
```

32. Regex Performance

Regex engines may perform backtracking.

Poorly designed patterns can become extremely slow.

Dangerous nested quantifier

```
^(a+)+$
```

This can be very slow for long invalid input such as:

```
aaaaaaaaaaaaaaaaaaaaa!
```

Safer equivalent

Use:

```
^a+$
```

Python:

```
import re

pattern = r"^a+$"

print(bool(re.fullmatch(pattern, "aaa")))
print(bool(re.fullmatch(pattern, "aaa!")))
```

Output:

```
True
False
```

Avoid nested broad quantifiers

Avoid:

```
(.)*+
```

Avoid:

```
([A-Za-z]*)*
```

Avoid:

```
(a|aa)+
```

These patterns can create many possible backtracking paths.

Prefer specific character classes

Less specific:

```
".*?"
```

More specific:

```
"[^\"]*"
```

Restrict input length

```
import re

def valid_username(value):
    if len(value) > 16:
        return False

    return bool(re.fullmatch(
        r"[A-Za-z][A-Za-z0-9_]{2,15}",
        value,
    ))

print(valid_username("arko_123"))
```

Output:

```
True
```

Use atomic groups when appropriate

Python 3.11+:

```
import re
```



```
pattern = r"^(?>[A-Za-z]+)$"

print(bool(re.fullmatch(pattern, "Python")))
```

Output:

```
True
```

Atomic groups prevent the engine from reconsidering an already accepted match.

33. Common Mistakes

33.1 Forgetting raw strings in Python

Less clear:

```
pattern = "\\d+"
```

Preferred:

```
pattern = r"\d+"
```

Raw strings reduce escaping confusion.

33.2 Using `match()` instead of `search()`

```
import re

print(re.match(r"[0-9]+", "Age 25"))
```

Output:

```
None
```

`match()` checks only at the beginning.

Use:

```
import re

print(re.search(r"[0-9]+", "Age 25").group())
```

Output:

25

33.3 Using `search()` for validation

Incorrect approach:

```
import re

print(bool(re.search(r"[0-9]{4}", "Year 2026")))
```

Output:

True

This only proves that four digits appear somewhere.

Use:

```
import re

print(bool(re.fullmatch(r"[0-9]{4}", "Year 2026")))
```

Output:

False

33.4 Forgetting that capturing groups change

`findall()`

```
import re

print(re.findall(r"(cat|dog)s?", "cats dogs"))
```

Output:

```
['cat', 'dog']
```

Use a non-capturing group:

```
import re

print(re.findall(r"(?:cat|dog)s?", "cats dogs"))
```

Output:

```
['cats', 'dogs']
```

33.5 Forgetting to escape a dot

```
import re

print(bool(re.fullmatch(r"file.txt", "fileXtxt")))
```

Output:

```
True
```

The dot matches `X`.

Correct:

```
import re

print(bool(re.fullmatch(r"file\.txt", "fileXtxt")))
```

Output:

False

33.6 Directly inserting user input

Unsafe:

```
import re

user_input = "file.txt"

print(bool(re.search(user_input, "fileXtxt")))
```

Output:

True

Safe:

```
import re

user_input = "file.txt"
pattern = re.escape(user_input)

print(bool(re.search(pattern, "fileXtxt")))
```

Output:

False

33.7 Expecting Bash regex to support lookahead

This Python pattern works:

```
[0-9]+(?:=INR)
```

Bash `[[ =~ ]]` does not support lookahead.

Use another tool such as GNU `grep -P`, Perl, or Python.

33.8 Quoting the Bash regex variable

Recommended:

```
regex='^[0-9]+$'  
value="123"  
  
[[ $value =~ $regex ]] && echo "Matched"
```

Avoid:

```
[[ $value =~ "$regex" ]]
```

Quoting the complete right-hand side changes how Bash interprets it.

33.9 Using regex for complex structured data

Do not use regex as a complete parser for:

- JSON
- HTML
- XML
- YAML
- Complex CSV
- Python programs
- Arbitrarily nested parentheses

Example using JSON properly:

```
import json  
  
data = json.loads('{"name": "Arko", "age": 25}')
```

```
print(data["name"])
```

Output:

34. Quick Reference

Character matching

Pattern	Meaning
<code>a</code>	Literal <code>a</code>
<code>.</code>	Any character except newline
<code>\.</code>	Literal dot
<code>[abc]</code>	One of <code>a</code> , <code>b</code> , or <code>c</code>
<code>[^abc]</code>	Anything except <code>a</code> , <code>b</code> , or <code>c</code>
<code>[a-z]</code>	Lowercase ASCII letter
<code>[A-Z]</code>	Uppercase ASCII letter
<code>[0-9]</code>	ASCII digit
<code>\d</code>	Digit in Python
<code>\w</code>	Word character
<code>\s</code>	Whitespace

Positions

Pattern	Meaning
<code>^</code>	Beginning
<code>\$</code>	End
<code>\A</code>	Absolute beginning in Python
<code>\Z</code>	Absolute end in Python
<code>\b</code>	Word boundary
<code>\B</code>	Non-word boundary

Repetition

Pattern	Meaning
<code>*</code>	Zero or more
<code>+</code>	One or more
<code>?</code>	Zero or one
<code>{3}</code>	Exactly three
<code>{2,}</code>	At least two
<code>{2,5}</code>	Between two and five
<code>*?</code>	Lazy zero or more
<code>+?</code>	Lazy one or more
<code>*+</code>	Possessive zero or more
<code>++</code>	Possessive one or more

Groups

Pattern	Meaning
<code>(abc)</code>	Capturing group
<code>(?:abc)</code>	Non-capturing group
<code>(?P<name>abc)</code>	Named group
<code>\1</code>	First numbered backreference
<code>(?P=name)</code>	Named backreference
<code>(?>abc)</code>	Atomic group

Lookarounds

Pattern	Meaning
<code>(?=abc)</code>	Positive lookahead
<code>(?!abc)</code>	Negative lookahead
<code>(?<=abc)</code>	Positive lookbehind
<code>(?<!abc)</code>	Negative lookbehind

Python functions

```
re.search(pattern, text)
re.match(pattern, text)
re.fullmatch(pattern, text)
re.findall(pattern, text)
re.finditer(pattern, text)
re.split(pattern, text)
re.sub(pattern, replacement, text)
re.subn(pattern, replacement, text)
re.compile(pattern)
re.escape(text)
```

Final Tool Selection

Use Python regex when you need:

```
Complex validation
Named groups
Lookahead and lookbehind
Advanced replacements
Overlapping matches
Atomic groups
Possessive quantifiers
```

Use Bash `[[ =~ ]]` when you need:

```
Simple validation
Shell-condition matching
Capturing a few values
```

Use `grep` when you need:

```
Searching lines
Extracting matching text
Recursive source-code searches
```

Use `sed` when you need:

Replacing text
Reformatting values
Deleting or transforming lines

Use `awk` when you need:

Regex with fields
Counting matches
Calculations
Log analysis
Structured text reports

The most important principle is:

Use regex to identify text structure, but use proper programming logic or dedicated parsers to validate complex meaning.